DATA STRUCTURES
EC-222

Lec. Kaynat Rana

LECTURE
01

COURSE OUTLINE

- **Introduction to Object Oriented Programming and C++**
- **Introduction to Data Structures**
- **Abstraction and ADT's**
- **Algorithm Analysis**
- **Built-in Data Structures in C++**
- **Linked Lists**
- **Stack**

COURSE OUTLINE

- **Queue**
- **Recursion**
- **Tree**
- **Sorting**
- **Searching**
- **Graphs**
- **Hashing**

MARKS DISTRIBUTION

Quiz	7.5%
Assignments	7.5 %
Midterm Exam	22.5 %
Finals	37.5 %
Lab Marks	25 %

Array Operations

Topics of the day

- What is an array
- Array creation and initialization
- Array Basic operations

Array

An array is a data structure that contains a group of elements. Typically these elements are all of the same data type, such as an integer or string.

Arrays are commonly used in computer programs to organize data so that a related set of values can be easily sorted or searched.

Array

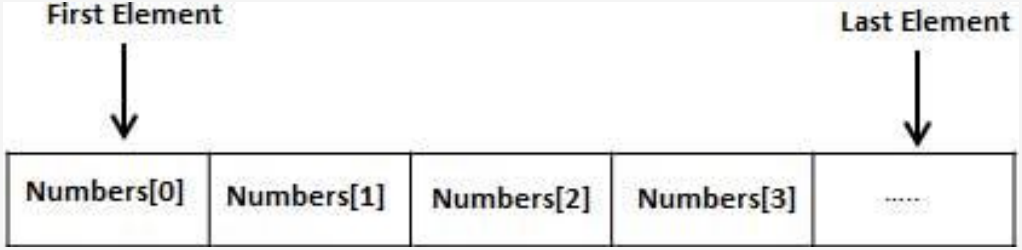
- Array is a container which can hold a fix number of items and these items should be of the same type.
- Most of the data structures make use of arrays to implement their algorithms.

Array

- **Element** – Each item stored in an array is called an element. □
- **Index** – Each location of an element in an array has a numerical index, which is used to identify the element.

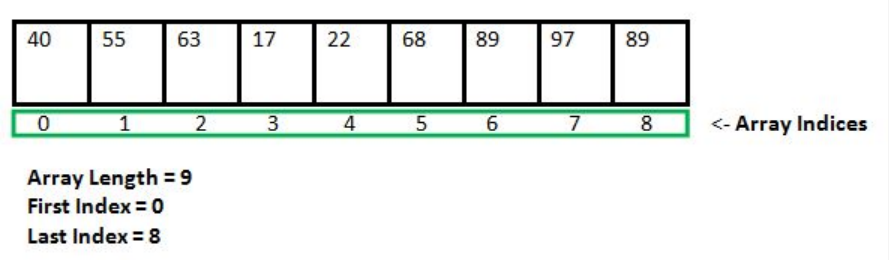
Diagrammatical

- Array



Array Creation

- Arrays can be declared in various ways in different languages
  - type var-name[];



```
int intArray[];
byte byteArray[];
short shortsArray[];
boolean booleanArray[];
long longArray[];
float floatArray[];
double doubleArray[];
char charArray[];
```

Array initialization

- When an array is declared, only a reference of array is created.
- To actually create or give memory to array, to use *new* to allocate an array, **you must specify the type and number of elements to allocate.**
  - int Array[]; //declaring array
  - Array = new int[20]; // allocating memory to array

Array initialization

- The elements in the array allocated by *new* will automatically be initialized to **zero** (for numeric types), **false** (for boolean), or **null** (for reference types).
- Obtaining an array is a two-step process. First, you must declare a variable of the desired array type. Second, you must allocate the memory that will hold the array, using new, and assign it to the array variable.

Default Values

Data Type	Default Value
bool	false
char	0
int	0
float	0.0
double	0.0f
void	
wchar_t	0

A Code Example

```
#include<iostream>
using namespace std;
int main(){
int a[10]={1,2,3,4,5,6,7,8,9,11};
for(int i=0;i<10;i++){
        cout<<a[i];
}}
```

Topics of the day

- What is an array
- Array creation and initialization
- Array Basic operations

Array Basic Operations

Traverse

Insertion

deletion

updating

search

Array traverse

- The word "traverse" means "to go or travel across or over"
- You can then print all the array elements one by one. or process the each element one by one
- Let A be a collection of data elements stored in the memory of the computer. Suppose we want to print the content of each element of A or suppose we want to count the number of elements of A with given property. This can be accomplished by traversing A, that is, by accessing and processing (frequently called visiting) each element of Array exactly once.

Array traverse algorithm

Step 1 : [Initialization] Set I = LB
Step 2 : Repeat Step 3 and Step 4 while I < = UB
Step 3 : [processing] Process the A[I] element
Step 4 : [Increment the counter] I = I + 1
[End of the loop of step 2]
Step 5: Exit

(*Here LB is lower Bound and UB is Upper Bound* A[ ] is linear array)

Array Basic Operations

Traverse

Insertion

deletion

updation

search

Array insertion

- insert operation is to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. □ Here, we see a practical implementation of insertion operation, where we add data at the end of the array –

Array insertion algorithm(last location)

- Let A IS THE ARRAY OF SIZE MAX OR N
- 1----start
- 2----if UB== Max-1
- Print overflow and exit
- 3----Read DATA
- 4----UB=UB+1
- 5----A[UB]=DATA
- 6---- Stop

Array insertion algorithm (start location)

- Let A IS THE ARRAY OF SIZE MAX OR N
- 1----start
- 2----if UB== Max-1
- Print overflow and exit
- 3----Read DATA
- 4----UB=k
- 5----repeat step 6 7 while K>=LB
- 6---- A[k+1]=A[k}

Array insertion algo(any location)

- Let A IS THE ARRAY OF SIZE MAX OR N
- 1----start
- 2----if UB== Max-1
- Print overflow and exit
- 3----Read DATA
- 4----Read LOC
- 5----UB=k
- 5----repeat step 6 7 while K>=LOC
- 6---- A[k+1]=A[k}

Array Basic Operations

Traverse

Insertion

deletion

updation

search




Array deletion

Write Down on a piece of Paper.

Array deletion

> Set ITEM=LA[K] □
> Repeat for J=K to N-1: □
> [Move J+1st element upward]
> Set LA[J]:= LA[J+1] □
> [end of loop] □
> Reset the number N of elements in LA]
> Set N=N-1 □ exit